

Supplementary Material: A Worked Example

Object-Centric Privacy Measures

Ignacio Marco-Pérez Christian Rennert Aaron Küsters Beatriz Pérez
Angel L. Rubio Wil van der Aalst

0. Scope and How to Read This

This is the supplementary material accompanying *Object-Centric Privacy Measures*, by Ignacio Marco-Pérez, Christian Rennert, Aaron Küsters, Beatriz Pérez, Angel L. Rubio and Wil van der Aalst. The paper formalizes k -anonymity, l -diversity and t -closeness as anonymity policies checked against OCPQ query results (Definitions 1–14 of its Section 4), implements the formalization as the **kanon** module, and evaluates it on the BPI Challenge 2017 log. This document grounds that formalization in a single worked example (one query, one policy, and every intermediate value the **kanon** module computes between the two, from the query’s binding box down to the final **AnonReport**) small enough that every number below can be checked by hand. Readers arriving from a citation in the paper may want:

Paper	Here
Section 4 (Anonymity Framework), Definitions 1–14	§2–§5 (concrete instances)
Section 5 (Implementation), “a policy modeled on P2.3”	this document, in full
Table 2, row P2.3	§6

A more detailed, definition-by-definition correspondence is in Section 7.

1. The Query: $\mathbf{b}_{Q6}$

The synthetic OCED models a small loan-application process: object type **Offer** and event types **0_Created** and **0_Accepted**, each offer created once and (in this data) always accepted once.¹ Fig. 1 shows the neighborhood of one **Application** object as OCPQ itself visualizes it. The binding box below, $\mathbf{b}_{Q6}$, uses only the offer/creation/acceptance portion of that graph, reusing the variable and type structure of query Q6 of the OCPQ evaluation² without reproducing that query’s original constraints or semantics, exactly as the paper does for all three reused binding boxes (its Section 5).

With object variable $y \mapsto \{\mathbf{Offer}\}$ and event variables $e_c \mapsto \{\mathbf{0_Created}\}$, $e_a \mapsto \{\mathbf{0_Accepted}\}$,

$$\mathbf{b}_{Q6} = (\{y, e_c, e_a\}, \{ \mathbf{E2O}(e_c, y, *), \mathbf{E2O}(e_a, y, *) \}).$$

The synthetic OCED contains 18 events and 13 objects in total; of these, 6 offers each have exactly one creation and one acceptance event, so $|\mathbf{out}(\mathbf{b}_{Q6})| = 6$. We write the six **0_Created** events as $\mathbf{oc1}, \dots, \mathbf{oc6}$; these are the elements the policy of Section 2 protects.

¹Synthetic OCED: https://github.com/igmarco/OCPQ-Anonymity/blob/main/backend/kanon_demo/data/bpic2017-ocel2-synth.json.

²OCPQ evaluation queries Q1, Q6, Q7: <https://github.com/aarkue/ocpq-eval>.

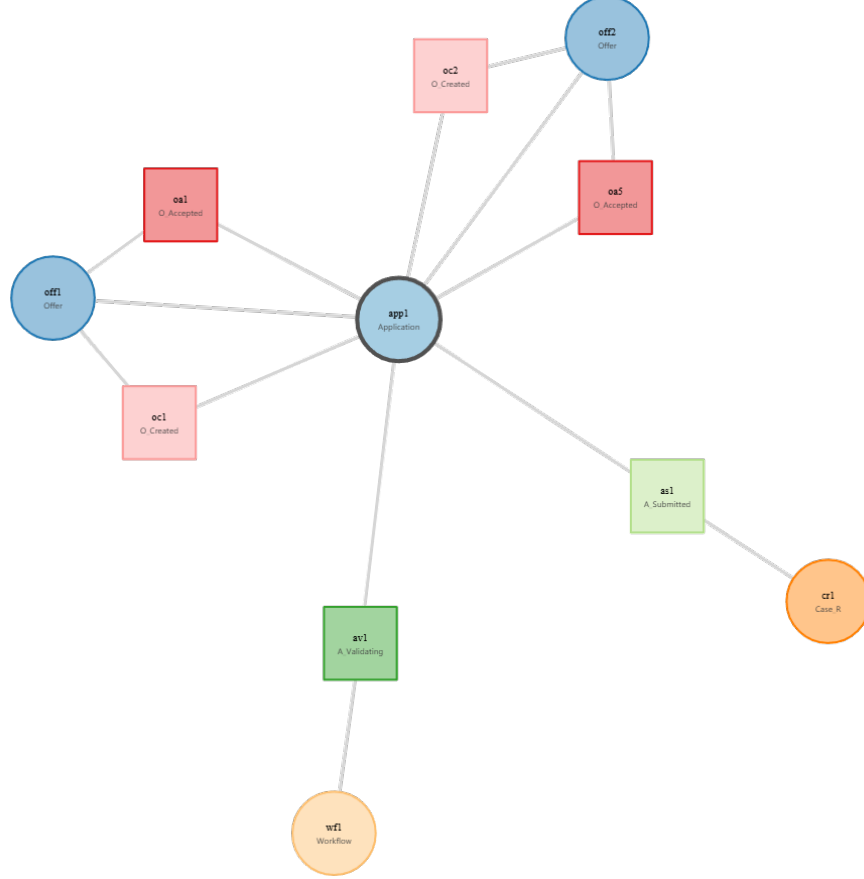


Figure 1: The neighborhood of one **Application** object (**app1**) in the synthetic OCED, as OCPQ itself visualizes it. Two **Offer** objects (**off1**, **off2**) each relate to their own **O.Created** and **O.Accepted** events; the application also relates directly to an **A.Submitted** and an **A.Validating** event, each in turn related to a **Case_R** or **Workflow** object. $\mathbf{b}_{Q6}$ below uses only the offer/creation/acceptance portion of this graph; the rest is shown only for context.

2. The Anonymity Policy: P

Both QIDs of P share a single pattern, the restriction of $\mathbf{b}_{Q6}$ to y and e_c :

$$\mathbf{p}_{\text{self}} = (\{y, e_c\}, \{\text{E2O}(e_c, y, *)\}).$$

This is itself worth noting as a case the general framework allows but the paper’s illustrative example does not show: nothing in Definition 3 (Quasi-identifier) requires distinct QIDs of a policy to use distinct patterns.

$$q_1 = (\mathbf{p}_{\text{self}}, e_c, e_c, \text{ApplicationType}), \quad q_2 = (\mathbf{p}_{\text{self}}, e_c, e_c, \text{LoanGoal}).$$

Both are *self*-QIDs: protected variable and source variable coincide ($v_{\text{prot}}^q = v_q = e_c$), so each QID assumes the adversary already knows one of the protected event’s own attributes: **ApplicationType** (New credit / Limit raise, 2 values among these six offers) for q_1 , **LoanGoal** (Car / Home improvement, 2 values among these six offers) for q_2 .

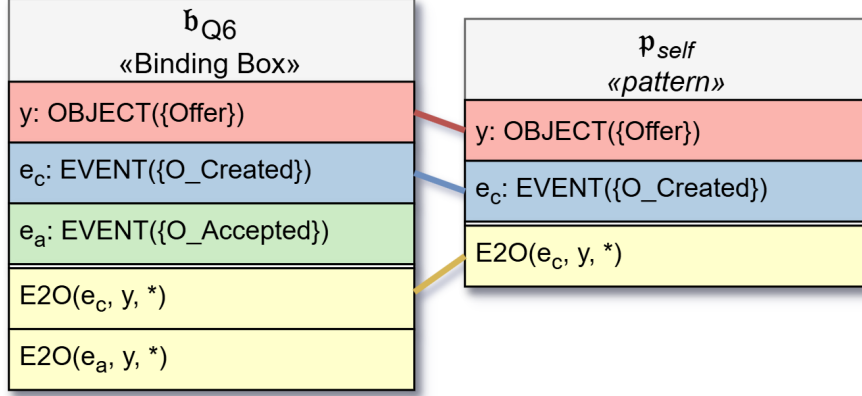


Figure 2: $\mathbf{b}_{Q6}$ (left) and the pattern $\mathbf{p}_{\text{self}}$ (right), in the paper’s binding-box notation, with the matching φ of this section drawn in: $\varphi(y) = y$ (red), $\varphi(e_c) = e_c$ (blue), and the predicate correspondence $\text{E2O}(e_c, y, *) \sim \text{E2O}(e_c, y, *)$ (yellow). e_a , present only on the left, plays no role in φ .

$$P = (\{q_1, q_2\}, \{\text{OfferedAmount}\}, k, l, t),$$

based on the two-QID structure of experiment P2.3 (Table 2 of the paper); Section 6 says precisely how P relates to, and differs from, that published experiment; in particular, why q_2 reads directly off e_c here rather than off a related `O_Accepted` event, as P2.3’s own q_2 does. We fix $k = 2$, $l = 2$, $t = 0.9$ purely as illustrative thresholds, enough to produce both a satisfied and a violated outcome below; a policy needs *some* values for k, l, t to be a policy (Definition 4), but what actually characterizes this data is the threshold-independent limit analysis of Section 5 ($k_{\max}, l_{\max}, t_{\min}$), not these particular numbers.

3. Activation and Matching

$\mathbf{p}_{\text{self}}$ matches into $\mathbf{b}_{Q6}$ via the injective map $\varphi(y) = y$, $\varphi(e_c) = e_c$ (leaving e_a unmapped): both variables carry compatible types, and $\text{E2O}(e_c, y, *)$ is present verbatim in $\mathbf{b}_{Q6}$ ’s predicate set, so $\varphi(\mathbf{p}_{\text{self}}) \sim \text{itself}$ (Definition 7). Fig. 2 shows this correspondence directly. This is the only matching for either QID, so $\text{Match}(\mathbf{p}_{\text{self}}, \mathbf{b}_{Q6}) = \{\varphi\}$ and q_1, q_2 are both activated: 2/2, the same activation count as the published P2.3 row (Section 6), though here both QIDs happen to share the very same matching rather than two independent ones.

4. From Real Values to Equivalence Classes

The six protected elements, with the two QID attributes and the sensitive attribute read directly off each:

o	ApplicationType	LoanGoal	OfferedAmount
oc1	New credit	Car	10000
oc2	New credit	Car	12000
oc3	New credit	Home improvement	8000
oc4	Limit raise	Home improvement	9000
oc5	Limit raise	Car	5000
oc6	Limit raise	Home improvement	6000

Because q_1 and q_2 are self-QIDs, their source sets are trivial: for every $o \in \{\text{oc1}, \dots, \text{oc6}\}$, $\text{Src}_{q_1}(o) = \{(o, \text{aval}_L(o, \text{ApplicationType}))\}$ and likewise for q_2 , a single pair, since the only matching’s only active binding maps e_c to o itself. The set-based deduplication that matters in the paper’s package/item example (its Definition 9) never triggers here: with one source and one matching, there is nothing to deduplicate. The marginal fingerprints are correspondingly one-element bags, e.g. $fp_{q_1}(\text{oc1}) = [\text{New credit}]$, $fp_{q_2}(\text{oc1}) = [\text{Car}]$, giving $fp(\text{oc1}) = ([\text{New credit}], [\text{Car}])$, and analogously for the other five.

Grouping the six elements by equal fingerprint gives four equivalence classes:

$$\{\text{oc1}, \text{oc2}\}, \quad \{\text{oc3}\}, \quad \{\text{oc4}, \text{oc6}\}, \quad \{\text{oc5}\}.$$

Individually, neither attribute is very identifying: **ApplicationType** and **LoanGoal** each take only 2 values among these six elements. But the *combination* is: (New credit, Home improvement) occurs only at **oc3**, and (Limit raise, Car) only at **oc5**, even though New credit, Home improvement, Limit raise and Car each individually recur elsewhere in the table. This is the same combinatorial phenomenon the paper’s BPIC 2017 evaluation reports at much larger scale in experiment P1.3 (its Section 5); two low-cardinality attributes are enough to single out an element once enough of them are combined, whether that means six elements here or twenty thousand there.

5. The Output: What AnonReport Actually Returns

Running `check_policy` on P and $\mathfrak{b}_{Q6}$ ’s output gives:

Field	Value
<code>policy_activated</code>	<code>true</code>
<code>activated_qid_ids</code>	$[q_1, q_2]$
<code>k_satisfied</code> (for $k = 2$)	<code>false</code> ($k_{\max} = 1$)
<code>l_satisfied</code> (for $l = 2$)	<code>false</code> ($l_{\max} = 1$)
<code>t_satisfied</code> (for $t = 0.9$)	<code>true</code> ($t_{\min} = 0.833$)
<code>elements_violating_k</code>	$\{\text{oc3}, \text{oc5}\}$
<code>sensitive_values_at_risk</code>	$\text{OfferedAmount} \in \{5000, 8000\}$

Both singleton classes fail k -anonymity outright ($|\{\text{oc3}\}| = |\{\text{oc5}\}| = 1 < 2$), and since each is also a single sensitive value, they fail l -diversity the same way (distinct-value count $1 < 2$): a class too small to protect k is, here, too homogeneous to protect l as well, though in general the two can fail independently. t -closeness holds throughout: with **OfferedAmount** distinct across all six elements, the global distribution is uniform over six values, and the worst-case per-class discrete EMD (attained by the two singleton classes, each placing all its mass on one value absent from the rest of the class) is $1 - \frac{1}{6} = 0.833 \leq 0.9$; the two size-2 classes are closer still, at 0.667.

6. Relation to the Published P2.3 Experiment

This section is about the paper, *Object-Centric Privacy Measures*, a separate, already-written document; nothing here changes what it reports, and every number in this section that is not restated above is the paper’s own, not recomputed here. P above is *based on* the two-QID structure of experiment P2.3, listed in the paper’s Table 2, same binding-box shape ($\mathbf{b}_{Q6}/Q6$), same two-QID and $k/l/t$ skeleton, but it is not a re-run of that experiment. P2.3 reads **EventOrigin** (self, from e_c) and **Action** (from the related **0_Accepted** event) on the full BPI Challenge 2017 log, protecting 42,995 elements. Both attributes happen to be constant across the six offers of the small synthetic OCED used in this document, which is why P instead reads **ApplicationType** and **LoanGoal**, both directly off e_c ; giving up the two-different-sources shape of P2.3’s q_2 (Section 2) and of the paper’s own illustrative package/item example (its Section 4), in exchange for a policy that actually partitions this small dataset into more than one class.

Numerically, the paper’s Table 2 reports $k_{\max} = 17,228$, $l_{\max} = 1$, $t_{\min} = 0.000$ for P2.3; P here gives $k_{\max} = 1$, $l_{\max} = 1$, $t_{\min} = 0.833$ (Section 5). The shared $l_{\max} = 1$ is reached by two structurally different routes: one enormous, sensitive-value-homogeneous class in the real log, against two singleton classes in the six-element synthetic one; both fail l -diversity, for opposite reasons. Knowing that both routes exist matters for anyone deciding what threshold to require: a low $l_{\max}$ alone does not say whether the underlying risk is a single huge non-diverse group or a handful of directly singled-out elements, and the two call for different remediation.

7. Cross-Reference Table

Paper reference	Content	Here
Definitions 1–4	Pattern, QID, sensitive attr. set, policy	§2
Definitions 5–8	Predicate compat., matching, active binding set, activation	§3
Definitions 9–10	Source set/marginal fingerprint, fingerprint	§4
Definition (equiv. class)	Fingerprint equality partitions the protected type	§4
Definitions 11–13	k -anonymity, l -diversity, t -closeness	§4–§5
Table 1 (kanon module)	<code>compute_source_set</code> , <code>compute_fingerprints</code> , <code>check_policy</code> , AnonReport	§4–§5
Section 5, “a policy modeled on P2.3”	This document	§1–§7
Table 2, row P2.3	Same query/policy shape, different data and attributes	§6